hyper2kvm vs virt-v2v

Why hyper2kvm Is the Superior Migration Platform

A detailed comparison of capabilities, architecture, and real-world use cases. Both tools convert VMs to KVM — but hyper2kvm delivers a complete migration platform where virt-v2v provides only a conversion utility.

Executive Summary

virt-v2v is a respected conversion tool. hyper2kvm builds on that foundation and adds everything needed for enterprise migration at scale.

10
Cloud providers supported
(virt-v2v: 0 cloud sources)

2
Deployment targets: KVM + KubeVirt
(virt-v2v: KVM only)

0
Dollars — Apache 2.0 license
(virt-v2v: also free, GPL)

80%
Less manual work per VM
(automated pipeline vs CLI flags)

The Core Difference

virt-v2v is a *single-step conversion utility* — it converts a disk image from one format to another and applies basic guest OS fixes. Every other migration task (export, deploy, verify, schedule) is left to the operator.

hyper2kvm is a *full migration platform* — it handles the entire pipeline: source discovery, export, conversion, offline guest repair, libvirt domain generation, KubeVirt manifest creation, deployment, and boot verification — all from a single command or interactive TUI.

Feature Comparison

Head-to-head capability matrix.

Capability	hyper2kvm	virt-v2v
VMware vSphere export	✓ Built-in govc integration	Requires manual export or libvirt URI
Hyper-V / VHD(X) support	✓ Native	✓
Cloud VM import (AWS, Azure, GCP…)	✓ 10 cloud providers via hypersdk	× Not supported
Disk conversion (VMDK, QCOW2, RAW)	✓ VMCraft + qemu-img	✓ libguestfs
Offline guest OS fixes	✓ VMCraft (pure Python + qemu-nbd)	✓ libguestfs (C + appliance)
Windows VirtIO injection	✓ Registry + driver install	✓
fstab / initramfs repair	✓ Automatic detection	✓
Libvirt domain XML generation	✓ Automatic, customizable	× Manual post-step
KubeVirt / Kubernetes deploy	✓ Built-in manifest + apply	× Not supported
VM boot verification	✓ Automated boot test	× Not supported
Interactive TUI	✓ zkvm (Go + Bubble Tea)	× CLI only
YAML-driven automation	✓ Declarative config	× CLI flags only
Batch / queue migration	✓ Job scheduler via hypersdk	× One VM at a time
Live progress streaming	✓ Real-time with ETA	× Minimal output
Pre-flight validation	✓ h2kvmctl doctor	× Not available
Air-gap / offline operation	✓	✓

No libguestfs dependency	✓ VMCraft is pure Python	× Requires libguestfs appliance

Use Case Advantages

Real-world scenarios where hyper2kvm delivers value that virt-v2v cannot.

1. VMware Exit at Scale

hyper2kvm: Discovers vSphere inventory, batch-exports VMs, converts, fixes, deploys to KVM or KubeVirt — one command per wave. Progress dashboard in TUI.
virt-v2v: Requires manual OVA export from vCenter, then run virt-v2v per VM, then manually create libvirt XML, then manually define + start. No batch, no progress.

2. Cloud Repatriation

hyper2kvm: Export VMs from AWS, Azure, GCP, OCI, or 6 other clouds via hypersdk, convert to QCOW2, deploy to on-prem KVM. Unified workflow.
virt-v2v: No cloud source support at all. Operator must manually download cloud VM images, then run virt-v2v. No automation for the hardest part.

3. KubeVirt / Cloud-Native Migration

hyper2kvm: Converts VM and generates KubeVirt VirtualMachine manifest. Applies to any K8s cluster (K3s, vanilla, EKS, OpenShift). Single pipeline.
virt-v2v: No KubeVirt support. Operator must manually create DataVolume, VirtualMachine YAML, upload disk image. Requires deep K8s expertise.

4. Operator-Friendly Migration

hyper2kvm: zkvm TUI guides operators step-by-step with form fields, file browser, source auto-detection, and real-time logs. No CLI expertise needed.
virt-v2v: CLI-only with complex flag combinations. Operators must memorize syntax like `-i ova -o local -of qcow2 -os /path`. Error messages are cryptic.

5. Windows Server Migration

hyper2kvm: Full pipeline — VirtIO injection, registry fixes, BSOD prevention, boot verification. Operator confirms VM boots before cutover.
virt-v2v: VirtIO injection works, but no boot test. If the fix didn't work, you find out only when you try to start the VM. Risky for production SQL Server.

6. Edge / Remote Site Deployment

hyper2kvm: Lightweight — no libguestfs appliance needed. VMCraft engine is pure Python + qemu-nbd. Works on minimal Linux installs at edge locations.
virt-v2v: Requires libguestfs + supermin appliance (500MB+). Difficult to install on minimal or hardened edge systems. Package dependency chain is heavy.

Architecture Advantage

Why hyper2kvm's design delivers more flexibility and reliability.

hyper2kvm Architecture

- **Two-layer design:** h2kvmctl (converter) + hypersdk (orchestrator)
- **VMCraft engine:** Pure Python + qemu-nbd — no C dependencies, no appliance boot
- **Modular pipeline:** Export → Convert → Fix → Deploy → Verify
- **YAML-driven:** Declarative configs, version-controlled, CI/CD ready
- **Dual target:** Same pipeline outputs libvirt XML and/or KubeVirt manifests
- **Interactive TUI:** zkvm provides guided workflow with real-time feedback
- **Hexagonal architecture:** Easy to add new source/target providers
- **Job scheduling:** Queue migrations, carbon-aware scheduling, cost estimation

virt-v2v Architecture

- **Monolithic:** Single OCaml binary, all-or-nothing conversion
- **libguestfs dependency:** Requires supermin appliance boot (~30s overhead per VM)
- **Single step:** Converts disk + applies fixes. No export, deploy, or verify
- **CLI flags:** Complex flag combinations, no config files
- **KVM only:** Outputs raw/qcow2 disk. No KubeVirt, no manifests
- **No UI:** Command-line only, no guidance for operators
- **Tightly coupled:** Adding new providers requires modifying OCaml source
- **No scheduling:** Run one VM at a time, no queue or batch support

The libguestfs Problem

virt-v2v depends on libguestfs, which boots a small Linux appliance (supermin) to mount guest filesystems. This adds **~30 seconds overhead per VM**, requires **500MB+ of packages**, and breaks on hardened systems with restricted kernel modules. hyper2kvm's VMCraft engine uses **qemu-nbd** to mount disks directly — no appliance, no boot, no overhead. Works on minimal Linux installs including edge devices and containers.

Migration Pipeline Comparison

What an operator does to migrate 50 VMware VMs to KVM.

#	Step	hyper2kvm	virt-v2v
1	**Discover VMs**	**vSphere tab in zkvm — browse datacenter, select VMs**	**Manual: log in to vCenter, identify VMs, note paths**
2	**Pre-flight check**	**`h2kvmctl doctor` validates all dependencies**	**No equivalent — discover missing deps at runtime**
3	**Export from vSphere**	**Built-in govc export with streaming progress**	**Manual OVA download from vCenter UI or ovftool**
4	**Convert disk**	**Automatic — part of pipeline**	**`virt-v2v -i ova -o local`**
5	**Guest OS fixes**	**Automatic — fstab, initramfs, VirtIO, network**	**Automatic — similar scope**
6	**Generate domain XML**	**Automatic — optimized libvirt XML**	**Manual: write XML or use virt-install**
7	**Deploy to libvirt**	**Automatic: virsh define + optional start**	**Manual: `virsh define` + troubleshoot**
8	**Boot verification**	**Automatic boot test confirms VM starts**	**Manual: start VM, hope it works**
9	**KubeVirt deploy**	**Automatic manifest generation + kubectl apply**	**Not supported — entirely separate workflow**
10	**Batch remaining VMs**	**Queue all 50, monitor in TUI**	**Repeat steps 3-8 for each VM manually**

Operator Time per VM

hyper2kvm: ~5 minutes of operator attention (select VM, review, confirm)
virt-v2v: ~45 minutes of manual work (export, convert, write XML, define, test, fix, retry)

For 50 VMs: **hyper2kvm saves ~33 hours** of operator time per migration wave.

When to Use Each Tool

Honest assessment of the right tool for the right job.

Use hyper2kvm When...

- Migrating 10+ VMs from VMware, Hyper-V, or any cloud
- You need KubeVirt / Kubernetes deployment
- Operators need a guided TUI experience
- You want automated boot verification
- You need batch migration with progress tracking
- YAML-driven automation for CI/CD integration
- Edge sites with minimal Linux (no libguestfs)
- Cloud repatriation from any provider
- You want one tool for the entire pipeline
- Enterprise features: scheduling, cost estimation, carbon-aware

Use virt-v2v When...

- Converting a single VM image already on disk
- You only need disk format conversion + basic fixes
- Already in a RHEL/Fedora environment with libguestfs
- Simple one-off conversion with no deployment needs
- You prefer a minimal, single-purpose tool

Note: hyper2kvm can also handle all virt-v2v use cases, plus much more.

Summary: hyper2kvm Is virt-v2v + Everything Else

virt-v2v solves one piece of the migration puzzle: disk conversion with guest fixes. hyper2kvm solves the **entire puzzle** — discovery, export, conversion, repair, deployment, verification, and orchestration. It's the difference between a screwdriver and a complete workshop.

Dimension	hyper2kvm	virt-v2v
Scope	Full migration platform	Disk conversion utility
Sources	VMware, Hyper-V, 10 clouds, local	VMware, Hyper-V, local
Targets	KVM + KubeVirt + OpenShift	KVM only
UI	CLI + TUI + YAML	CLI only
Dependencies	Python + qemu-nbd (lightweight)	libguestfs + supermin (heavy)
Automation	YAML configs, batch, scheduling	Shell scripts (DIY)

Verification	Automated boot test	None
License	Apache 2.0	GPL v2+